

COMP20290



Improving Counting Sort Algorithm Via Data Locality

Group Members

Anna Curry	24304011
Carrie Savage	24354821
Jewel Hicks	24221876
Julie Seery	24415002

Semester 2 2026
Dr. Fatemeh Golpayegani
University College Dublin

Introduction

Sorting algorithms are widely regarded as one of the foundational constructs within computer science, their significance extending deeply into professional and applied domains. In practice, sorting appears in database indexing, file system organisation, network packet scheduling and operating system process management. Even marginal performance gains can carry meaningful real world impact at scale.

While the theoretical time complexities of sorting algorithms are well understood, their practical performance is heavily influenced by hardware factors, for example CPU cache behaviour and memory capacity. This report describes our implementation and evaluation of the paper "*Improving Counting Sort Algorithm Via Data Locality*" written by Mahmud, Haque and Choudhury (ACMSE 2022). The paper proposes a cache-friendly hybrid sorting algorithm that combines Counting Sort with a modified Quicksort preprocessing step.

The central insight of the paper is that although the Counting Sort algorithm runs with an $O(n + r)$ time complexity (where n is the number of elements and r is the value range), its practical performance degrades significantly for unsorted or random input. For random inputs, accesses to the count and output arrays occur in non-sequential order, causing frequent cache misses.

Our team implemented the proposed hybrid algorithm in Java, replicated key experiments from Tables 1–3, and analysed its performance and cache efficiency improvements. Our results broadly reflect the paper's findings, the hybrid approach consistently outperforms the classic Counting Sort for large random inputs and demonstrates competitive execution timings against Quicksort variants. The remainder of the report covers the proposed methods, our implementation decisions, the experimental method, results and a critical discussion of our findings relative to the paper.

Proposed Method : Cache Friendly Combined Counting Sort

The paper introduces a hybrid sorting algorithm designed to improve the real-world performance of Counting Sort by addressing its poor memory design. The proposed algorithm operates in two phases. In the **preprocessing phase** a modified Quicksort partitions the input array into subarrays satisfying the CPU cache constraint (**maxValue - minValue + high - low**) $\leq C$, without fully sorting them. Counting Sort is then applied independently to each subarray, now guaranteed to produce cache friendly access patterns, because the subarrays are ordered relative to each other, the output is globally sorted.

Implementation Details

Classical Counting Sort - countingSort(int[] arr, int n)

The countingSort method is implemented accurately according to Algorithm 1 in the paper. The helper function getMax() calculates the count[] array size ($r+1$ to account for 0-based indexing). Occurrences are counted, prefix sums computed and output[] is filled by iterating

in reverse for stability. The sorted output is copied back into the original array, serving as the baseline with no cache optimisation.

Modified Quicksort Preprocessing - quicksortModified(int[] arr, int low, int high, int maxValue, int minValue)

This is the core of the proposed algorithm. The method implements algorithm 2 from the paper with the following behaviours :

- The stopping condition $(\text{maxValue} - \text{minValue} + \text{high} - \text{low}) > C$ is checked at the start of each recursive call via a while loop. If the condition is not met, the method returns immediately, leaving that subarray to be sorted by Counting Sort in phase 2
- If the condition is met, partition() is called to rearrange arr[low ... high] around a pivot and return the final pivot index. The value at that index is recorded
- Two recursive calls are then made, one on the left subarray with maxValue updated to midValue (since all values in the left partition are less than or equal to the pivot) and one on the right subarray with minValue updated to midValue (since all values in the right partition are greater than or equal to pivot)
- Critically, the method returns immediately after the two recursive calls. This means the while loop condition is only evaluated once per invocation, the algorithm relies entirely on recursion to process subarrays not on loop iteration

Partition Function - partition(int[] arr, int low, int high)

Our partition() method implements the Hoare partition scheme with a median of three pivot selection, as specified in the paper. The implementation completes in two stages :

Stage 1 - Pivot Selection

The midpoint is computed. Three conditional swaps are then applied to sort arr[low], arr[mid] and arr[high] into increasing order relative to each other. After these three swaps, arr[mid] holds the median of the three values which becomes the pivot. This approach avoids the $O(n^2)$ worst case time complexity of classical quicksort on already-sorted or nearly sorted input arrays, which can arise in practice during recursive calls on partially ordered data.

Stage 2 - Hoare Partitioning

With the pivot value set, two pointers are initialised $i = \text{low} - 1$ and $j = \text{high} + 1$. The algorithm enters an infinite loop, i is incremented until $\text{arr}[i] \geq \text{pivot}$ and j is decremented until $\text{arr}[j] \leq \text{pivot}$ (both using do while loops to guarantee at least one step). If $i \geq j$ the loop exists and j is returned as the partition boundary. Otherwise $\text{arr}[i]$ and $\text{arr}[j]$ are swapped to move out of place elements to their correct side.

Partition Counting Sort - countingSortByPartitions(int[] arr, int n)

This method applies counting sort on a per-partition basis rather than over the entire array, enabling the cache locality benefit described in the paper. Starting from index 0, it scans forward to determine the extent of each partition using the same cache threshold C . For each partition, it tracks a running partMin and partMax. If adding the next elements would trigger the cache restraint, the scan stops and countingSortSegment() is called on the segment using the known max and min values. The outer loop then advances to the next unprocessed segment. The private helper countingSortSegment() performs counting sort on a single segment. It allocates a count array of size $(\text{max} - \text{min} + 1)$, counts occurrences of

each value offset by min, then writes the sorted values back into arr[low ... high] in a single pass.

Additional Sorting Algorithms for Comparison

The class also includes a recursive quicksort and a hybrid quicksort with insertion sort for use in the Table 3 comparisons from the paper. Both share the same partition() method as the modified quicksort ensuring consistent pivot selection behaviour across all three sorts.

Challenges Encountered

Three significant challenges arose during the implementation of CountingSortAlgorithm, each of which required analysis of both the paper's pseudocode and the runtime behaviour of the java implementation.

Understanding what quicksortModified() actually sorts

The most difficult aspect of the paper was grasping the quicksortModified() as it is deliberately an incomplete sort. Our initial assumption was that quicksortModified() would fully sort the array. Testing revealed it only partitions the data, and re-reading the paper clarified that countingSort() is intentionally applied per-partition to achieve the cache locality benefit.

Infinite recursion with arrays containing many duplicate values

Arrays with many repeated values caused the program to hang and throw a StackOverflowError. When all elements equal the pivot, Hoare partitioning splits the array into roughly equal halves but both recursive calls then receive a subarray where maxVal - minVal = 0. With the stopping condition depending entirely on high - low, large equal element subarrays exceed the threshold $C = 1000$, causing infinite recursion. We fixed this by adding an early exit at the top of quicksortModified(), if maxVal == minVal all elements are identical so the method returns immediately and leaves that block to countingSort().

Applying counting sort on each partition instead of on the whole array

We initially called countingSort() once over the entire array after preprocessing, this was correct in output but missed the paper's core idea. We corrected this by implementing countingSortByPartitions(), which applies countingSortSegment() to each partition using its local max and min, properly achieving the cache locality benefit.

Experiments

We implemented all experiments on an Apple M1 MacBook Air, which has 192KB of L1 instruction cache and 128KB of L1 data cache per performance core, with a shared 12MB L2 cache. We conducted three sets of experiments directly replicating the paper's Tables 1, 2, and 3. All benchmarks were implemented in Java using System.nanoTime() for timing, with results converted to milliseconds. Input arrays were populated with uniformly distributed random integers in the range $[0, r-1]$, consistent with the paper's description and randomly drawn integers.

A fixed random seed (21) was used throughout for reproducibility. While a different seed might cause minor timing fluctuations, it would not change our conclusion, as the speedup is driven by hardware cache behaviour rather than specific input values.

Experiment 1 - Performance comparison over random and sorted inputs

The first experiment (Table 1) measured the performance of classic counting sort on random versus pre-sorted input. No preprocessing step was applied in either case. The expectation is that sorted input runs approximately 2-3 times faster than unsorted input.

Experiment 2 - Performance comparison of counting sort with and without preprocessing

The second experiment (Table 2) evaluated the benefit of the modified quicksort preprocessing step in the case where $r > n$. The preprocessing time and the subsequent counting sort time were measured and reported separately, then summed to give the total hybrid time, which was compared against plain counting sort with no preprocessing.

Experiment 3 - Comparison of quicksort, quicksort with insertion sort and the hybrid algorithm

The third experiment (Table 3) compared three algorithms classic quicksort, a quicksort insertion sort hybrid, and the proposed modified quicksort with counting sort under the ideal case of $n = r$. For each trial, a fresh copy of the input array was passed to each algorithm to ensure identical input across comparisons.

Results

The three tables below present our results obtained from the experiments described above. Original paper values are shown in parentheses where available for comparison. Our results show both expected and unexpected outcomes. We analyse each result in detail.

Table 1 - Random VS Sorted Input

Expected : Sorted input should be approximately 2-3x faster due to cache locality

n	r	Random (ms)	Sorted (ms)	Speed Increase
1,000,000	1,000,000	8.353 (1.09)	6.195 (0.47)	1.35x
2,000,000	2,000,000	10.936 (2.84)	6.309 (0.91)	1.73x
3,000,000	3,000,000	25.222 (3.75)	10.113 (1.39)	2.49x

Table 1 confirms that classic counting sort runs significantly faster on sorted input than on random input, with speedup ratios of 1.35x, 1.73x and 2.49x, averaging 1.86x across all three cases. Although this falls slightly below the paper's predicted 2-3x range at smaller input sizes, the trend aligns with the cache locality effect described.

Both sets of results support the same conclusion, unsorted inputs cause the `count[]` and `output[]` arrays to be accessed in a random order, increasing cache misses and producing a measurable impact on performance.

Table 2 - Hybrid Counting Sort Algorithm VS Plain Counting Sort

Expected Result : Preprocessing + Counting Sort < Plain Counting Sort

n	r	Preprocess (ms)	Counting Sort (ms)	Total hybrid (ms)	Plain (ms)
1,000	1,000,000	0.053	0.646	0.700	0.860
2,000	1,000,000	0.053	0.308	0.361	0.890
3,000	1,000,000	0.079	0.424	0.503	0.896

Table 2 shows that the hybrid algorithm is consistently faster than plain countingSort() across all three cases, aligning with the paper's expectation that preprocessing improves performance.

Preprocessing times are consistently small across all three cases, meaning the preprocessing step contributes negligible overhead. The dominant benefit comes from the improved cache behaviour of the countingSort() phase following preprocessing.

The particularly low runtime observed at $n = 2000$ can be attributed to cache effects. At this input size, the working data may fit more efficiently within the CPU's L1 or L2 cache allowing faster memory access during the counting phase.

Table 3 - Comparison of Quicksort + Quicksort & Insertion Sort + Hybrid Algorithm

Expected : Proposed Hybrid < Quicksort + Quicksort + Insertion sort

n = r	Quicksort (ms)	Quicksort + insertion sort (ms)	Proposed hybrid (ms)
1,000,000	75.10	69.08	68.98
2,000,000	159.78	142.83	137.82

Table 3 confirms that our proposed hybrid algorithm outperforms classical quicksort. While the improvement ratios of 8% and 14% are more modest than the papers reported 25%, the direction and scaling of the result are consistent with its predictions.

The ordering of Hybrid < Quicksort + Insertion < Classical Quicksort is preserved exactly as predicted at both input sizes. At $n = 1,000,000$ quicksort with insertion sort and the hybrid algorithm perform nearly identical. The gap widens at $n = 2,000,000$ indicating that the hybrid's cache locality advantage becomes more meaningful as input size grows.

Discussion

Our results largely support the paper's findings across all three tables. Table 1 establishes the fundamental idea of the paper, cache locality as a driver of counting sort performance. The results confirmed that sorted input is faster than random input under classic Counting Sort, though our speedups were smaller, peaking at 2.49x compared to the paper's 3.1x. This is likely due to hardware differences between our machine and the paper's Intel i7-7500U, but the direction of the result is the same and still clearly demonstrates the data locality effect that motivates the hybrid algorithm.

Table 2 was the most cleanly positive result. The hybrid outperformed plain counting sort in all three cases with very small preprocessing overhead, which directly supports the paper's claim that partitioning the input into cache-sized subarrays before sorting is worthwhile even when r is much larger than n .

Table 3 was the most encouraging result, as our findings matched the paper's ordering exactly. The proposed hybrid was the fastest at both input sizes, and the advantage over classic quicksort grew from around 8% at $n = 1,000,000$ to around 14% at $n = 2,000,000$. This suggests that the cache-locality benefit of the hybrid scales well with input size, which aligns with the paper's argument, giving us confidence that our implementation is correct and the algorithm behaves as the paper describes.

Our Java execution times are considerably higher than the paper's C++ values across all tables. This is expected as Java programs carry overhead that C++ does not, including JVM startup costs. Crucially, these factors slow down all variants equally so while they do raise execution times they do not distort the speedup ratios, the core finds therefore remain valid regardless of the language difference.

The most notable discrepancy between our results and the paper's appears in Table 3. Our improvement ratios of 8% and 14% over classical Quicksort fall below the papers reported 25%, the same hardware and language factors discussed above account for the magnitude difference.

Conclusion

This report covered our implementation and evaluation of the cache-friendly hybrid sorting algorithm proposed by Mahmud, Haque and Choudhury (2022). The algorithm preprocesses input into cache-sized partitions using a modified Quicksort before applying Counting Sort to each partition individually, reducing cache misses and improving real-world performance on large random inputs. All three experiments validated the paper's findings. Table 1 confirmed that sorted input produces measurably faster Counting Sort results due to cache locality. Table 2 showed the hybrid consistently outperformed plain Counting sort with negligible preprocessing overhead. Table 3 confirmed the hybrid outperformed both Quicksort variants, with the advantage widening at larger input sizes.

The main limitations of the algorithm are the loss of stability of counting sort due to the quicksort preprocessing step, and the threshold C (combined size of value range and number of elements in subarray before the modified quicksort stops recursing) is hardware-dependent rather than universally applicable. As well as this additional preprocessing introduces overhead for small inputs. Future work could look at using a stable sorting algorithm in the preprocessing step and finding a more adaptive way to set the threshold C .

In summary, the hybrid algorithm succeeds in its core objective. Preprocessing the input into cache sized partitions meaningfully reduces cache misses during the counting sort phase producing consistent performance gains that grow with input size.